

Ashley Klahm Review Doc! - Computer Graphics!

Highlighted -> Discussed/ right answer

What is the rendering pipeline?

The rendering pipeline is the series of steps used in turning a 3D scene into a 2D image on a screen

IS not DOES* ^^^**

PROCESS CPU -> GPU Displays to screen(Frame Buffer)

In and out

Components of the typical rendering pipeline/ their roles:

Geometry: the shape of the objects being rendered

(where it is) Vertex Shader: Where it is/ the depth

(Colour) Fragment Shader: What color is it, shading?

Output: Like polish it up and make it 2D (?)

Series of connected blocks.

Vertex Shader (Placement of vertices)

Fragment Shader

Stencil Buffer

Illustrate the rendering pipeline using figure:

Vertices _> GEOMETRY -> VERTEX/FRAGMENT -> **FRAME BUFFER** (raterization?) _>
OUTPUT(pixels on screen) (?)

Example of each component rendering pipeline done:

Vertices: finding the position on the vertices

Geomtery: connecting the vertices to create the shapes

Vertex shader: Depth and place in space

Provide examples directly to topics seen:

What are forward and deferred rendering ?

Forwards = Geo -> vertex -> geo -> frag _> fram buffer

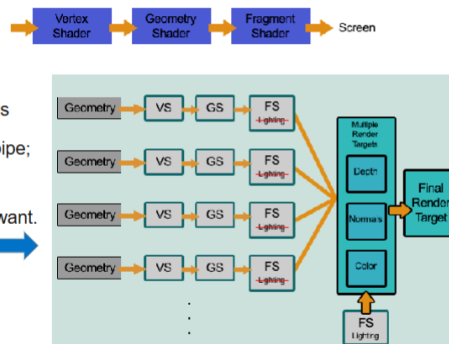
Deferred = Geo -> Vertex -> Geo -> frag -> G BUFFER -> lighting _> frame buffer

Applications of forward and deferred rendering?

LECTURE 8****

Deferred Rendering

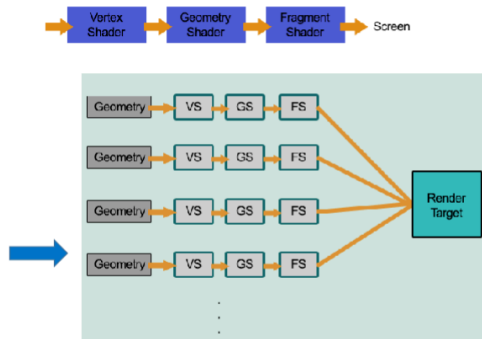
- ❖ In deferred rendering the rendering is deferred a little bit until all of the geometries have passed down the pipe; the final image is then produced by applying shading **at the end**.
- ❖ We can have as many lights as we want.
- ❖ Performance is optimal.
- ❖ No limits to shadows.
- ❖ It does not cause additional passes.



Forward Rendering

- ❖ Remember the graphics pipeline..

- ❖ Keep in mind that forward rendering is **linear**, and each geometry is passed down the pipe one at a time to produce the **final image**.



Deferred = more efficiency,, stored in G-buffer, light is rendered as geometry

G-buffer

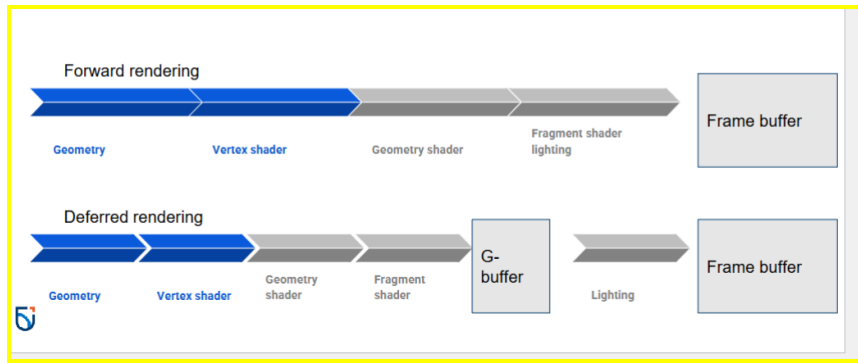
Forward vert mod _> frag _> frame buffer

Per-pixel lighting each pixel is color individually (complex)

Per-Vertex lighting - cheaper and easier

Rendering each time forward rendering - step by step and add at end

Differed - Send everything to G-Buffer and than do it all together - more efficient.



Render Doc(?) analysis

Vertex Shader: Depth and position of vertices/objects in the scene
 Fragment Shader: The colour and shading of the objects in the scene

Vertex is used for creating depth, movement(?) Looks- depending on position, height maps
 Fragment is used for colours, textures, applying shading etc. lighting texturing.

Diagrams: In -> Vertex/Fragment -> out

Vertex Shader, scrolling one
 Fragment; ambient